

FOODWINGS BACKEND GUIDE

Complete Architecture, File-by-File Breakdown & Step-by-Step Learning Guide

Stack: FastAPI (Python) • PostgreSQL 16 (psycopg3 Pool) • Plain JavaScript Frontend • JWT Auth

1. System Architecture & Mental Model

FoodWings operates as a unified multi-role food delivery platform. Instead of creating 4 separate microservices for customers, restaurants, delivery partners, and admins, FoodWings employs a **Clean Modular Monolith** design. All 4 interfaces interact with a single high-performance FastAPI backend connected to a shared PostgreSQL database.

Role / Component	Key Responsibilities & Flow	Key Tables
1. Customer	Browse restaurants, view menus, manage cart, place orders, make card/COD payments, track delivery in real-time.	users, restaurants, menu_items, orders, cart_items
2. Restaurant	Onboard new restaurant, toggle store Open/Closed, manage live incoming orders (Accept / Reject / Food Ready).	restaurants, orders, order_status_history
3. Rider (Partner)	Register & upload verification docs, toggle Online/Offline, receive nearby order assignments, accept/pickup/deliver.	delivery_partners, partner_documents, delivery_assignments
4. Admin	Approve or reject restaurant partner registrations and rider document verifications.	restaurants, partner_documents, user_roles
5. Mock Gateway	Simulates Stripe/Razorpay credit card tokenization, authorization, capture, and webhooks safely.	payment_transactions, payment_methods

2. Backend Directory Structure Explained

File Path	Category	Detailed Purpose & What It Does
config.py	App Configuration	Uses Pydantic BaseSettings to read environment variables (PostgreSQL connection URI, JWT secrets, OTP expiry, Rate limits, Mock gateway flags). Everything is strictly typed and centralized.
db.py	Connection Pool & Transactions	Manages high-concurrency PostgreSQL connection pooling with psycopg_pool.ConnectionPool. Features the critical transaction(conn) context manager that ensures ACID multi-table atomic rollbacks upon any error.

<code>logging_setup.py</code>	PCI-DSS Redaction Filter	Security enforcement layer: intercepts Python log records and replaces any 16-digit credit card number or 3-digit CVV with asterisks (****) before it is written to disk or console. Prevents compliance violations.
<code>security.py</code>	JWT Tokens & Role Auth	Handles token generation (<code>create_token</code>), user context extraction (<code>current_user</code>), cryptographic SHA-256 OTP hashing (<code>hash_otp</code>), and FastAPI role-checking dependency injection (<code>require_roles</code>).
<code>schemas.py</code>	Pydantic Input Validation	Strict request validation schemas using Pydantic v2. Validates phone formats (<code>^[+]?[0-9]{10,14}\$</code>), 6-digit OTPs, addresses, cart items, onboard forms, and payment requests before they ever hit the database.
<code>errors.py</code>	Global Exception Handlers	Catches all Unhandled Exceptions, validation errors, and HTTPExceptions, formatting them into standard JSON responses with proper HTTP status codes, error messages, and meta tracking IDs.
<code>mock_gateway.py</code>	Payment Gateway Simulator	Provides full Razorpay/Stripe mock APIs for testing. Converts card inputs into secure one-time tokens, simulates success/decline card numbers, signs webhooks via HMAC-SHA256, and handles refunds.
<code>services/gateway.py</code>	Gateway Client Abstraction	Decouples the core application from specific payment processors. Offers a standard interface (<code>create_order</code> , <code>charge</code> , <code>refund</code>). Sanitizes gateway payloads (Layer 8 security rule) to discard unvetted card keys.
<code>services/order_flow.py</code>	Order State Machine & Dispatch	Encapsulates order transitions (<code>change_status</code>) with history logging and automatic geospatial delivery partner dispatch (<code>offer_to_nearest_partner</code>) based on distance and availability.
<code>main.py</code>	Single-File Unified API	The master controller! Houses all route definitions, database transactions, request routing, static file mounting, and lifespan management. Grouped logically into 11 sections for maximum readability.

3. Deep Dive: Key Architectural Patterns

A. Passwordless Authentication & Role Multi-Tenancy

FoodWings does not use passwords. Users authenticate via an OTP sent to their phone number. A single human can have multiple roles (e.g. they can be a Customer AND a Delivery Partner).

- **OTP Security:** Raw OTPs are NEVER saved in the database. `security.hash_otp(phone, otp)` computes a SHA-256 digest with salt. Only the hash is saved in `otp_requests`.
- **Single-Use Consumption:** `consume_otp(conn, phone, otp, purpose)` verifies the hash, checks expiry (5 mins), limits attempts (max 3), and marks `used_at = now()` in a single transaction.
- **Unified User Table:** The `users` table stores the user profile, while `user_roles` connects a user ID to multiple roles (1=Customer, 2=Owner, 3=Staff, 4=Rider, 5=Admin).

B. Database Connection Pooling & Safe Transactions

Under `api/app/db.py`, we manage a connection pool configured for high concurrency:

- **Autocommit Mode:** Reads run fast without leaving open transaction locks.
- **Atomic Writes:** Every multi-table mutation uses `with transaction(conn):`. If any step fails (e.g. stock deduction fails after order creation), PostgreSQL automatically rolls back the entire operation.

C. PCI-DSS Compliance & Card Data Isolation

A critical real-world architectural design implemented in FoodWings:

- **Zero Card Storage:** Card numbers and CVVs never touch the application database.
- **Client-side Tokenization:** The frontend sends card data directly to the Mock Gateway, which issues a safe token (`tok_xxxx`). Only this token is sent to the backend.
- **Logging Redaction:** `logging_setup.py` intercepts all log records using regex filters to mask card and CVV digits.

4. Breakdown of main.py (The 11 Sections)

Section	Title & Routes	Core Logic & Explanation
Section 1	Setup, Logging & Lifespan	Initializes FastAPI, configures CORS middleware, registers startup/shutdown hooks to open/close the database pool and load mock gateway tokens.
Section 2	Security & Role Checking	Contains helper functions to extract JWT from Authorization header and enforce role checks (e.g. requiring role 2 for restaurant actions).
Section 3	Authentication (/auth/*)	POST /auth/otp/request, POST /auth/check-phone, POST /auth/register, POST /auth/login, POST /auth/refresh, and GET /me. Handles registration and login across all 3 apps.
Section 4	Customer & Menu (/restaurants, /cart)	GET /restaurants (filter open stores), GET /restaurants/{id}/menu (nested category items), POST /me/addresses, and full Cart management (GET/POST/DELETE).

Section 5	Order & Checkout (/orders/*)	POST /orders/quote (calculates taxes, packaging, delivery fee, coupons), POST /orders/checkout (creates order in PENDING state), GET /me/orders (history & tracking).
Section 6	Payments (/payments/*)	POST /payments/create-intent (creates gateway order), POST /payments/confirm (charges card token, transitions order to PAID/PLACED), POST /payments/webhook.
Section 7	Restaurant Owner (/restaurant/*)	POST /restaurants/onboard (onboards new restaurant), GET /restaurant/me, POST /restaurant/open-toggle, GET /restaurant/orders, and POST /restaurant/orders/{id}/status.
Section 8	Delivery Partner (/partner/*)	POST /partners/onboard, POST /partner/online, POST /partner/location (GPS updates), GET /partner/available-orders, POST /partner/orders/{id}/accept/pickup/deliver.
Section 9	Admin Operations (/admin/*)	POST /admin/restaurants/{id}/approve, POST /admin/partners/{id}/verify-docs. Allows admin to review and activate pending restaurants and drivers.
Section 10	Mock Gateway (/mock-gateway/*)	Routes mock payment transactions for development and automated testing without requiring live third-party accounts.
Section 11	Health & Static Serving	GET /health (verifies DB connection) and mounts the frontend HTML/JS files to serve the complete application on a single port.

5. Step-by-Step Lifecycle of an Order

Step 1: Browsing & Cart

Customer selects restaurant, chooses dishes, and adds items to cart. The backend verifies item availability and pricing.

Step 2: Quote & Checkout

POST /orders/quote computes item total + GST (5%) + packaging + delivery fee - discounts. POST /orders/checkout locks prices and creates order with status PLACED.

Step 3: Payment Confirmation

Customer pays via MockCard / UPI. POST /payments/confirm charges token and records transaction. Order moves to PAYMENT_SUCCESS.

Step 4: Restaurant Acceptance

Restaurant owner sees live order in dashboard, clicks Accept -> status becomes ACCEPTED. Kitchen prepares food -> status becomes PREPARING.

Step 5: Rider Auto-Assignment

When food is ready (READY_FOR_PICKUP), order_flow.py finds nearest online partner with no active order and creates assignment.

Step 6: Pickup & Delivery

Rider accepts order, arrives at restaurant, collects food (PICKED_UP), navigates to customer address, and marks DELIVERED.

6. How to Study, Understand, and Present This Project

Use this 4-step framework when reviewing code or explaining this system in interviews or presentations:

1. Trace Requests from Route to DB

Start in main.py at any endpoint (e.g. /auth/register). Follow the Pydantic schema in schemas.py, see how the database connection is acquired via get_conn(), observe the with transaction(conn): block, and see the final returned JSON dictionary.

2. Understand Data Ownership & Roles

Notice how JWT claims (user_id and roles) protect each section. A customer cannot call restaurant endpoints because require_roles(2, 3) checks their role ID.

3. Run Automated Tests with Pytest

Run pytest -q to see 12 comprehensive integration tests validating auth, registration, concurrency, role isolation, and payments. Read tests/test_auth_register_login.py as living documentation!

4. Key Concepts to Mention in Interviews

Highlight: **Connection Pooling** (efficient DB scaling), **ACID Transactions** (no orphan data), **PCI-DSS Compliance** (tokenized payments without card liability), **Passwordless OTP Auth**, and **Clean Modular Monolith architecture**.